

ENAUTO Lesson 1 – Raspberry Pi Setup & Python Fundamentals

Objective

Build a clean Raspberry Pi automation server and complete the first Python lesson.

Lab Setup

Completed:

- Imaged Raspberry Pi OS Lite (64-bit)
- Updated the system:
`sudo apt update`
`sudo apt full-upgrade -y`
- Configured SSH
- Verified system health with:
`hostnamectl`
`free -h`
`df -h`
`ip addr`

Observed:

- Debian 13 (Trixie)
- Linux 6.18
- ARM64
- 8 GB RAM
- 118 GB storage
- IP: 192.0.2.10

Hostname: enauto-pi

SSH Key Authentication

Existing Ed25519 key pair on Windows was reused.

The public key should be copied into `~/.ssh/authorized_keys` on the Pi.

Avoid creating a new key pair if existing servers already trust the current key.

Python Environment

Installed development tools including Git, pip, python3-venv and utilities.

Created a virtual environment:

```
python3 -m venv enauto
source enauto/bin/activate
```

Purpose:

A virtual environment isolates project dependencies from the operating system.

First Program

hello.py

```
print("ENAUTO lab is running.")
```

Run with:

```
python hello.py
```

Lesson 1 - Variables

Variables store values in memory.

Example:

```
hostname = "enauto-pi"  
ip_address = "192.0.2.10"  
username = "student"  
ram = 7.4  
ssh_enabled = True
```

```
print(hostname)  
print(ip_address)  
print(username)  
print(ram, "GiB")  
print(ssh_enabled)
```

Concepts Learned

- Strings (str)
- Integers (int)
- Floating-point numbers (float)
- Booleans (bool)
- Variables
- print()
- Python executes code from top to bottom.

Mistakes and Lessons

Mistake

```
ram = 7.4 GiB
```

Lesson

Units are not part of a numeric literal.

```
ram = 7.4 "GiB"
```

Separate values need syntax to connect them.

```
ram = 7.4 ("GiB")
```

Parentheses indicate a function call.

```
ssh enabled
```

Variable names cannot contain spaces; use snake_case.

```
Edited with sudo
```

Files in your home directory should normally be edited without sudo.

Key Takeaways

The goal is not memorizing syntax. The goal is understanding how Python thinks.

Read error messages, make a hypothesis, test a fix, and repeat. That is how experienced engineers learn.

Future lessons will build toward network automation with APIs, Ansible, Netmiko, RESTCONF, and NETCONF.